JavaScript — Zero to Master

25 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (7 topics)

01 Introduction and Setup
02 Variables and Data Types
03 Operators and Type Coercion
04 Conditions and Loops
05 Functions
06 Arrays and Objects
07 DOM Basics

Intermediate (7 topics)

08 Array Methods (map, filter, reduce)
09 Scope, Hoisting and Closures
10 this, call, apply, bind
11 Objects, Prototypes and Classes
12 ES Modules
13 Error Handling
14 JSON, Storage and Dates

Advanced (7 topics)

15 Asynchronous JavaScript: Callbacks and Promises
16 async / await and fetch
17 Event Loop, Microtasks and Timers
18 Iterators, Generators, Symbols
19 Higher-Order Functions and Functional Style
20 Proxies, Reflect, WeakMap

21 Browser APIs

Master (4 topics)

22 Performance and Memory
23 Design Patterns and Clean Code
24 Testing and Tooling
25 Security Basics

Beginner Level

01. Introduction and Setup

DEFINITION

JavaScript is the programming language of the web; it runs in browsers and on servers (Node.js). Run code in the browser console, a `<script>` tag, or with `node file.js`.

EXAMPLE

```
console.log('Hello, JavaScript!');
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop notice board that changes the price instantly when the owner edits it; JavaScript changes a web page live without reloading.

தமிழ்

கடை விலைப் பலகையில் உரிமையாளர் மாற்றியவுடன் உடனே விலை மாறுவது போல, JavaScript பக்கத்தை மீண்டும் ஏற்றாமல் உடனே மாற்றுகிறது.

PROS

- Runs in every browser
- Huge ecosystem
- Also works on servers (Node.js)

CONS

- Loose typing causes bugs
- Many quirks to learn

ADVANCED / NEW FEATURES

- ES2025 features, Temporal API (modern dates) rolling out, native TypeScript-like type stripping in Node

02. Variables and Data Types

DEFINITION

Use `const` by default and `let` when a value changes; avoid `var`. Types: string, number, boolean, null, undefined, bigint, symbol, object.

SYNTAX

```
let name = value;
const name = value;
```

EXAMPLE

```
const name = 'Arun';
let age = 25;
console.log(typeof name, typeof age);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Steel boxes labelled 'rice' and 'oil' in a kitchen: `const` is a sealed box, `let` is a box whose content you may replace.

தமிழ்

சமையலறையில் 'அரிசி', 'எண்ணெய்' என்று பெயரிட்ட டப்பாக்கள்: `const` மூடி போடப்பட்ட டப்பா, `let` உள்ளடக்கத்தை மாற்றக்கூடிய டப்பா.

PROS

- `let/const` are block scoped
- `const` prevents accidental reassignment

CONS

- `var` hoisting surprises
- Dynamic types allow mistakes

ADVANCED / NEW FEATURES

- `using` declarations (resource management) proposal
- Strict mode is default in modules

03. Operators and Type Coercion

DEFINITION

Use `===` (strict equality) instead of `==`. Falsy values: `false, 0, '', null, undefined, NaN`. Use `??` for defaults and `?.` for safe access.

SYNTAX

```
a === b
value ?? defaultValue
obj?.prop
```

EXAMPLE

```
console.log(0 == '0', 0 === '0'); // true false
const city = user?.address?.city ?? 'Unknown';
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

At a bank, your name and your account number must both match; `===` checks value and type, `==` is a careless clerk who guesses.

தமிழ்

வங்கியில் பெயர், கணக்கு எண் இரண்டும் பொருந்த வேண்டும்; `===` மதிப்பு, வகை இரண்டையும் சரிபார்க்கும்; `==` ஊகித்துச் சொல்லும் கவனக்குறைவான எழுத்தர் போன்றது.

PROS

- Short-circuit operators keep code compact
- `??` and `?.` avoid crashes

CONS

- Coercion rules are confusing
- `NaN !== NaN`

ADVANCED / NEW FEATURES

- Logical assignment `||=`, `&&=`, `??=` (ES2021)

04. Conditions and Loops

DEFINITION

`if/else`, `switch`, `for`, `while`, `for...of` (values) and `for...in` (keys).

SYNTAX

```
if (cond) { } else { }
for (const x of list) { }
```

EXAMPLE

```
for (const n of [1, 2, 3]) {
  if (n % 2) console.log(n, 'odd');
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bus conductor checks each passenger: if has ticket continue, else issue a ticket; he repeats for every seat (loop).

தமிழ்

நடத்துநர் ஒவ்வொரு பயணியிடமும் டிக்கெட் இருந்தால் தொடர்கிறார், இல்லையென்றால் கொடுக்கிறார்; ஒவ்வொரு இருக்கைக்கும் இதை மீண்டும் செய்கிறார் (loop).

PROS

- Simple, familiar syntax
- `for...of` works with any iterable

CONS

- `for...in` on arrays is error-prone
- Infinite loops are easy to write

ADVANCED / NEW FEATURES

- `Array.prototype.at()`, `findLast()`
- Iterator helpers (ES2025)

05. Functions

DEFINITION

Declare with `function`, function expressions or arrow functions. Support default and rest parameters; functions are values and can be passed around.

SYNTAX

```
function name(a, b = 1, ...rest) { return value; }
const f = (a, b) => a + b;
```

EXAMPLE

```
const add = (a, b = 0) => a + b;
const sum = (...nums) => nums.reduce((t, n) => t + n, 0);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A juice machine: put fruit in (parameters), get juice out (return value). The same machine works for many fruits.

தமிழ்

ஜூஸ் இயந்திரம்: பழத்தைப் போடு (parameters), ஜூஸ் வரும் (return). ஒரே இயந்திரம் பல பழங்களுக்குப் பயன்படும்.

PROS

- First-class functions
- Concise arrow functions
- Reusable code

CONS

- `this` differs between arrow and normal functions
- Hoisting confuses beginners

ADVANCED / NEW FEATURES

- `Promise.withResolvers()` (ES2024)
- Decorators proposal

06. Arrays and Objects

DEFINITION

Arrays hold ordered lists (`push`, `pop`, `slice`); objects hold key-value pairs. Use destructuring and the spread operator to copy and unpack.

SYNTAX

```
const [a, b] = arr;
const { x, ...rest } = obj;
```

EXAMPLE

```
const [a, b] = [10, 20];
const { name, ...rest } = { name: 'Priya', age: 22, city: 'Madurai' };
const copy = { ...rest, age: 23 };
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shopping list is an array (ordered); a customer card is an object (name, phone, city as labelled fields).

தமிழ்

கடைப் பட்டியல் array (வரிசை); வாடிக்கையாளர் அட்டை object (பெயர், தொலைபேசி, ஊர் எனப் பெயரிட்ட புலங்கள்).

PROS

- Flexible data structures
- Destructuring and spread are expressive

CONS

- Mutating shared objects causes bugs
- Shallow copy pitfalls

ADVANCED / NEW FEATURES

- `structuredClone`
- `toSorted/toReversed/with` (ES2023)
- `Object.groupBy` (ES2024)

07. DOM Basics

DEFINITION

The DOM is the page as objects. Select elements with `querySelector`, change `textContent`/classes and react to events with `addEventListener`.

SYNTAX

```
document.querySelector(sel)
el.addEventListener('click', fn)
```

EXAMPLE

```
document.querySelector('#btn').addEventListener('click', () => {
  document.querySelector('#msg').textContent = 'Clicked!';
});
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A TV remote: pressing a button (event) makes the TV react. The DOM is the TV's internal structure; JavaScript presses and changes it.

தமிழ்

டிவி ரிமோட்: பொத்தானை அழுத்தினால் (event) டிவி பதிலளிக்கும். DOM என்பது டிவியின் உள் அமைப்பு; JavaScript அதைத் தொட்டு மாற்றுகிறது.

PROS

- Direct control of the page
- Works everywhere

CONS

- Manual DOM updates get messy at scale
- Slow if overused

ADVANCED / NEW FEATURES

- View Transitions API
- Popover and `<dialog>` APIs reduce JS

Intermediate Level

08. Array Methods (map, filter, reduce)

DEFINITION

Transform data without loops: `map` (transform), `filter` (keep), `reduce` (combine), `find`, `some`, `every`, `sort`.

SYNTAX

```
arr.map(fn)
arr.filter(fn)
arr.reduce(fn, init)
```

EXAMPLE

```
const total = [10, 20, 30].filter(n => n > 10).map(n => n * 2).reduce((t, n) => t + n, 0); // 100
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

In a mango market: `filter` keeps ripe mangoes, `map` packs each into a box, `reduce` counts the total weight.

தமிழ்

மாம்பழச் சந்தையில்: `filter` பழுத்தவற்றை வைத்திருக்கும், `map` ஒவ்வொன்றையும் பெட்டியில் அடைக்கும், `reduce` மொத்த எடையைக் கணக்கிடும்.

PROS

- Readable data pipelines
- No manual loops
- Immutable style

CONS

- Chaining creates intermediate arrays
- `reduce` can be hard to read

ADVANCED / NEW FEATURES

- `toSorted`, `toSpliced`, `findLast`
- `Array.fromAsync`
- `Object.groupBy`

09. Scope, Hoisting and Closures

DEFINITION

`let` / `const` are block scoped. A **closure** is a function that remembers variables from where it was created, enabling private state.

EXAMPLE

```
function counter() {
  let c = 0;
  return () => ++c;
}
const next = counter(); next(); next(); // 2
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A tea master remembers each regular's sugar preference even after they leave: a closure remembers the variables where it was created.

தமிழ்

தேநீர் மாஸ்டர் வாடிக்கையாளர் சென்ற பிறகும் அவரின் சர்க்கரை விருப்பத்தை நினைவில் வைத்திருப்பார்; closure தான் உருவான இடத்தின் மாறிகளை நினைவில் வைக்கிறது.

PROS

- Private state
- Powerful function factories
- Basis of modules and hooks

CONS

- Can cause memory leaks
- Loop-closure pitfalls with `var`

ADVANCED / NEW FEATURES

- Block-scoped `let` fixed classic loop closure bugs

10. this, call, apply, bind

DEFINITION

`this` depends on how a function is called. Arrow functions inherit `this` from outside. `bind` fixes `this` permanently.

SYNTAX

```
fn.call(obj, a)
fn.apply(obj, [a])
const g = fn.bind(obj)
```

EXAMPLE

```
const user = { name: 'Ravi', hi() { return 'Hi ' + this.name; } };
const f = user.hi.bind({ name: 'Meena' });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

'I' means a different person depending on who speaks. `this` points to whoever calls the function.

தமிழ்

'நான்' என்பது பேசுபவரைப் பொறுத்து மாறுவது போல `this` அந்தச் செயலியை அழைப்பவரைப் பொறுத்து மாறும்.

PROS

- Flexible method reuse
- `bind` fixes context

CONS

- Easily lost in callbacks
- Confusing for beginners

ADVANCED / NEW FEATURES

- Arrow functions and class fields reduce `bind` use

11. Objects, Prototypes and Classes

DEFINITION

Objects inherit through the prototype chain. `class` syntax is a cleaner way to write constructors, methods, `static` members and `extends`.

SYNTAX

```
class A extends B {
  #priv = 1;
  static x = 2;
  method() { }
}
```

EXAMPLE

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
}
class Dog extends Animal { speak() { return `${this.name} barks`; } }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A family: grandfather's land (prototype) passes to children who can add their own rooms (class inheritance).

தமிழ்

குடும்பத்தில் தாத்தாவின் நிலம் (prototype) பிள்ளைகளுக்கு வரும்; அவர்கள் தங்கள் அறைகளைச் சேர்க்கலாம் (class inheritance).

PROS

- Clean class syntax
- Real private fields with `#`
- Familiar to Java/C# developers

CONS

- Still prototype-based under the hood
- Deep inheritance is fragile

ADVANCED / NEW FEATURES

- Private `#` fields, static blocks (ES2022)
- Decorators (stage 3)

12. ES Modules

DEFINITION

Split code into files with `export` and `import`; modules have their own scope and are loaded once.

SYNTAX

```
export const x = 1;
export default fn;
import { x } from './a.js';
```

EXAMPLE

```
// math.js
export const add = (a, b) => a + b;
// app.js
import { add } from './math.js';
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A big kitchen split into stations (chutney, sambar, rice); each station exports only what others need.

தமிழ்

பெரிய சமையலறை சட்னி, சாம்பார், சாதம் எனப் பிரிவுகளாக இருக்கும்;
ஒவ்வொன்றும் மற்றவர்களுக்குத் தேவையானதை மட்டும் வழங்கும் (export).

PROS

- Encapsulation
- Tree shaking
- Native in browsers and Node

CONS

- CommonJS/ESM interop headaches
- Needs bundlers for older browsers

ADVANCED / NEW FEATURES

- Top-level `await`
- Import maps
- JSON import attributes
- `require(esm)` in Node

13. Error Handling

DEFINITION

Use `try/catch/finally` and throw `Error` objects. Always handle failures in asynchronous code.

SYNTAX

```
try { } catch (e) { } finally { }
throw new Error('msg');
```

EXAMPLE

```
try { JSON.parse('{bad json'); }
catch (e) { console.error('Invalid JSON:', e.message); }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

When the gas runs out while cooking, a prepared cook switches to the induction stove instead of abandoning the meal.

தமிழ்

சமைக்கும் போது கேஸ் தீர்ந்தால், தயாரான சமையல்காரர் உணவைக் கைவிடாமல் இண்டக்ஷன் அடுப்புக்கு மாறுவார் (error handling).

PROS

- Graceful failure
- Stack traces help debugging

CONS

- Overuse hides bugs
- Async errors need extra care

ADVANCED / NEW FEATURES

- `Error.cause`
- Optional catch binding
- `Promise.try` (new)

14. JSON, Storage and Dates

DEFINITION

`JSON.stringify/parse` convert data to text; `localStorage` stores strings in the browser; `Date` and `Intl` format dates and numbers.

SYNTAX

```
JSON.stringify(obj)
JSON.parse(text)
localStorage.setItem(k, v)
```

EXAMPLE

```
localStorage.setItem('user', JSON.stringify({ id: 1 }));
const u = JSON.parse(localStorage.getItem('user'));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Writing a customer's address on a slip (JSON) and keeping it in a drawer (localStorage) so you can read it tomorrow.

தமிழ்

வாடிக்கையாளர் முகவரியை சீட்டில் (JSON) எழுதி இழுப்பறையில் (localStorage) வைத்து நாளை படிப்பது போல.

PROS

- Simple persistence
- JSON is universal

CONS

- localStorage is synchronous and small (~5MB)
- Not secure for secrets

ADVANCED / NEW FEATURES

- `Intl` improvements
- Temporal API for dates

- `structuredClone`

Advanced Level

15. Asynchronous JavaScript: Callbacks and Promises

DEFINITION

Async work (network, timers) finishes later. A **Promise** represents a future value with states pending/fulfilled/rejected and is chained with `then/catch`.

SYNTAX

```
fetch(url).then(r => r.json()).catch(err => {})
```

EXAMPLE

```
fetch('/api/users')
  .then(r => r.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

You place a dosa order and get a token; you do other things and collect it when it is ready. A promise is that token.

தமிழ்

தோசை ஆர்டர் செய்து டோக்கன் பெறுகிறீர்கள்; வேறு வேலை பார்த்துவிட்டு தயாரானதும் வாங்குகிறீர்கள். Promise என்பது அந்த டோக்கன்.

PROS

- Avoids callback hell
- Composable (`all`, `race`)
- Standard

CONS

- Unhandled rejections crash silently
- Cannot cancel natively

ADVANCED / NEW FEATURES

- `Promise.allSettled`, `any`
- `Promise.withResolvers`
- `AbortSignal.timeout`

16. async / await and fetch

DEFINITION

`async/await` makes promises read like normal code. Use `Promise.all` to run tasks in parallel.

SYNTAX

```
async function f() {
  const r = await fetch(url);
}
```

EXAMPLE

```
async function load() {
  const [users, posts] = await Promise.all([
    fetch('/users').then(r => r.json()),
    fetch('/posts').then(r => r.json()),
  ]);
  return { users, posts };
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Reading a letter line by line in order, instead of jumping around: `await` pauses until the answer arrives so the code reads top to bottom.

தமிழ்

கடிதத்தை வரிசையாகப் படிப்பது போல `await` பதில் வரும் வரை காத்திருந்து, குறியீட்டை மேலிருந்து கீழாகப் படிக்கும்படி செய்கிறது.

PROS

- Reads like synchronous code
- Works with try/catch

CONS

- Accidentally sequential awaits are slow
- Needs `async` context

ADVANCED / NEW FEATURES

- Top-level `await` in modules
- `Array.fromAsync`

17. Event Loop, Microtasks and Timers

DEFINITION

JavaScript is single threaded. The event loop runs the call stack, then **microtasks** (promises), then **macrotasks** (timers). This explains output order.

EXAMPLE

```
console.log(1);
setTimeout(() => console.log(2));
Promise.resolve().then(() => console.log(3));
console.log(4); // 1 4 3 2
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

One cashier serves many customers: while a customer fetches change, the cashier serves the next. The event loop schedules waiting tasks without extra cashiers.

தமிழ்

ஒரே காசாளர் பல வாடிக்கையாளர்களுக்கு சேவை செய்கிறார்; ஒருவர் சில்லறை எடுக்கச் சென்றால் அடுத்தவரைக் கவனிக்கிறார். Event loop காத்திருக்கும் பணிகளை இப்படி நிர்வகிக்கிறது.

PROS

- Simple concurrency model
- No locks needed

CONS

- Long tasks block everything
- Order of microtasks/macrotasks confuses

ADVANCED / NEW FEATURES

- `queueMicrotask`
- `scheduler.postTask`
- Web Workers for heavy work

18. Iterators, Generators, Symbols

DEFINITION

Objects with `[Symbol.iterator]` work in `for...of`. **Generators** (`function*`) pause and resume with `yield`.

SYNTAX

```
function* gen() { yield 1; }
for (const v of gen()) { }
```

EXAMPLE

```
function* ids() { let i = 1; while (true) yield i++; }
const g = ids(); g.next().value; // 1
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A ration queue token machine gives numbers one at a time only when you press the button; a generator produces values on demand.

தமிழ்

ரேஷன் வரிசை டோக்கன் இயந்திரம் பொத்தானை அழுத்தும்போது மட்டும் ஒரு எண்ணைத் தரும்; generator தேவைப்படும்போது மட்டும் மதிப்புகளைத் தரும்.

PROS

- Lazy evaluation
- Memory efficient
- Custom iteration

CONS

- Harder to debug
- Not reusable once consumed

ADVANCED / NEW FEATURES

- Iterator helpers `.map().filter().take()` (ES2025)
- Async generators

19. Higher-Order Functions and Functional Style

DEFINITION

Functions that take or return functions: currying, composition, pure functions and immutability make code predictable and testable.

SYNTAX

```
const add = a => b => a + b;
const compose = (...fns) => x => ...
```

EXAMPLE

```
const compose = (...fns) => x => fns.reduceRight((v, f) => f(v), x);
const add = a => b => a + b;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A recipe machine where you can plug in different steps: wash, cut, cook. Higher-order functions accept steps as inputs.

தமிழ்

கழுவு, வெட்டு, சமை என வெவ்வேறு படிகளை இணைக்கக்கூடிய சமையல் இயந்திரம் போல higher-order function வேலைகளை உள்ளீடாக ஏற்கும்.

PROS

- Reusable logic
- Easy testing of pure functions

CONS

- Deep nesting reduces readability
- Possible performance overhead

ADVANCED / NEW FEATURES

- Pipeline operator (proposal)
- Iterator helpers

20. Proxies, Reflect, WeakMap

DEFINITION

`Proxy` intercepts object operations (validation, reactivity); `WeakMap` stores private data without preventing garbage collection.

SYNTAX

```
new Proxy(target, handler)
```

EXAMPLE

```
const p = new Proxy({}, { set(o, k, v) { if (k === 'age' && v < 0) throw Error('bad'); o[k] = v; return true; } });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A security guard at a gate who checks every visitor before letting them in; a Proxy intercepts reads and writes on an object.

தமிழ்

வாயிலில் ஒவ்வொரு பார்வையாளரையும் சோதிக்கும் காவலர் போல Proxy ஒரு object-இன் படிப்பு/எழுத்தை இடைமறிக்கிறது.

PROS

- Validation and reactivity
- Transparent wrappers

CONS

- Slower than plain objects

- Hard to debug

ADVANCED / NEW FEATURES

- Basis of Vue's reactivity
- `WeakRef` and `FinalizationRegistry`

21. Browser APIs

DEFINITION

Fetch, `IntersectionObserver`, Web Storage, `History`, Web Workers, WebSockets, Canvas and the Clipboard API power rich apps.

SYNTAX

```
new IntersectionObserver(cb).observe(el)
```

EXAMPLE

```
new IntersectionObserver(([e]) => e.isIntersecting && console.log('visible')).observe(el);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop camera that alerts you only when a customer enters the frame. Browser APIs notify you when something happens.

தமிழ்

கடை கேமரா வாடிக்கையாளர் வரும்போது மட்டும் எச்சரிக்கை தருவது போல browser API-கள் ஏதாவது நடந்தால் தெரிவிக்கின்றன.

PROS

- Powerful native capabilities
- No libraries needed

CONS

- Support differs between browsers
- Permissions required for some

ADVANCED / NEW FEATURES

- View Transitions
- Navigation API
- WebGPU
- Popover API

Master Level

22. Performance and Memory

DEFINITION

Avoid layout thrashing, debounce events, use `requestAnimationFrame`, lazy load code, and avoid memory leaks (forgotten listeners, timers, big closures).

EXAMPLE

```
const debounce = (fn, ms) => { let t; return (...a) => { clearTimeout(t); t = setTimeout(() => fn(...a), ms); }; };
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A traffic police avoids jams by regulating signals instead of letting everyone rush; debouncing and lazy loading regulate work.

தமிழ்

போக்குவரத்துக் காவலர் அனைவரையும் ஒரே நேரத்தில் செல்ல விடாமல் சிக்னல் மூலம் கட்டுப்படுத்துவது போல debounce, lazy loading வேலையைக் கட்டுப்படுத்துகின்றன.

PROS

- Faster pages
- Better user experience

CONS

- Premature optimisation wastes time
- Needs measuring tools

ADVANCED / NEW FEATURES

- `requestIdleCallback`
- `content-visibility`
- Speculation Rules API

23. Design Patterns and Clean Code

DEFINITION

Module, Observer, Factory, Singleton and Strategy patterns; keep functions small, name clearly, and avoid global state.

EXAMPLE

```
class Emitter {
  #h = {};
  on(e, f) { (this.#h[e] ??= []).push(f); }
  emit(e, d) { (this.#h[e] || []).forEach(f => f(d)); }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A kitchen has standard recipes (patterns) like 'tiffin combo' that many hotels reuse; design patterns are reusable solutions.

தமிழ்

பல உணவகங்கள் 'டிபன் காம்போ' போன்ற தரப்படுத்தப்பட்ட செய்முறைகளைப் பயன்படுத்துவது போல design patterns மீண்டும் பயன்படுத்தக்கூடிய தீர்வுகள்.

PROS

- Shared vocabulary
- Proven solutions

CONS

- Over-engineering
- Patterns misapplied add complexity

ADVANCED / NEW FEATURES

- Modern JS replaces some patterns (modules, classes, signals)

24. Testing and Tooling

DEFINITION

Test with Vitest/Jest, lint with ESLint, format with Prettier, bundle with Vite; use npm scripts and Git hooks.

EXAMPLE

```
npm create vite@latest
npm i -D vitest eslint prettier
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bus is checked by a mechanic and a fitness certificate before it runs. Linting, formatting and tests make code roadworthy.

தமிழ்

பேருந்து ஓடும் முன் மெக்கானிக் சோதனை, தகுதிச் சான்று தேவை. Lint, format, tests குறியீட்டைத் தகுதியாக்குகின்றன.

PROS

- Catches bugs early
- Consistent code style

CONS

- Setup time
- Tooling churn

ADVANCED / NEW FEATURES

- Vite, Vitest, Biome and Oxc (Rust tooling)
- Node built-in test runner

25. Security Basics

DEFINITION

Prevent XSS (never insert untrusted HTML), use HTTPS, validate input, avoid `eval`, and keep dependencies updated (`npm audit`).

EXAMPLE

```
el.textContent = userInput; // safe
// el.innerHTML = userInput; // dangerous
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop locks the cash box and checks every note; web apps must validate and escape everything users send.

தமிழ்

கடை பணப்பெட்டியைப் பூட்டி ஒவ்வொரு நோட்டையும் சரிபார்ப்பது போல வலைச் செயலிகள் பயனர் அனுப்பும் அனைத்தையும் சரிபார்த்து escape செய்ய வேண்டும்.

PROS

- Protects users and data

CONS

- Adds development effort
- Easy to miss edge cases

ADVANCED / NEW FEATURES

- Trusted Types
- Content Security Policy improvements

- `Sanitizer` API

Interview Question Bank

Q1. var vs let vs const?
`var` is function scoped and hoisted; `let` and `const` are block scoped; `const` cannot be reassigned.

Q2. What is a closure?
A function that keeps access to variables of its outer scope after the outer function has returned.

Q3. == vs ===?
`==` coerces types; `===` compares value and type without coercion.

Q4. Explain the event loop.
The call stack runs synchronous code; completed async callbacks wait in queues; microtasks (promises) run before macrotasks (timers).

Q5. What is `this`?
The object a function is called on; arrow functions use the surrounding `this`.

Q6. Promise vs async/await?
`async/await` is syntax sugar over promises for cleaner sequential code.

Q7. What is hoisting?
Declarations are processed before execution; `var` and functions are hoisted, `let/const` stay in the temporal dead zone.

Q8. Shallow vs deep copy?
Spread/`Object.assign` copy one level; use `structuredClone` for deep copies.

Q9. What is debouncing vs throttling?
Debounce waits until activity stops; throttle runs at most once per interval.

Q10. Prototype vs class?
Classes are syntax over prototypal inheritance.

Capstone Project Ideas

1. To-do app with localStorage
2. Weather app using a public API
3. Quiz game with timer
4. Expense tracker with charts
5. Markdown note-taking app